SurveyAI SaaS Hazirlik Degerlendirmesi

Kapsam

Bu dokuman, `SurveyAI` projesinin mevcut kod tabani incelenerek hazirlanmistir. Degerlendirme; SaaS urunlesme icin kritik olan alanlari kapsar:

- kimlik ve yetki yonetimi
- tenant/workspace yonetimi
- self-service onboarding
- abonelik, kullanim ve faturalama
- guvenlik ve uyumluluk
- operasyon, gozlemlenebilirlik ve release surecleri
- test, kalite ve dokumantasyon olgunlugu

Yonetici Ozeti

Proje bugun itibariyla "tek tenant veya yakin destekle yonetilen urun prototipi / internal beta" seviyesine daha yakin. Cekirdek urun mantigi guclu: anket yonetimi, operasyon/call-job akisleri, provider webhook entegrasyonlari, analytics ve company-scoped veri modeli mevcut. Ancak bunu dogrudan satilabilir bir SaaS urune donusturmek icin kritik eksikler var.

En buyuk bosluklar:

- self-service tenant olusturma ve kullanici yasam dongusu yok
- rol bazli yetkilendirme fiilen uygulanmiyor
- faturalama, plan, kullanim limiti ve tahsilat katmani yok
- production-grade secret management ve guvenlik varsayimlari hazir degil
- gozlemlenebilirlik, audit trail, CI/CD ve operasyonel guardrail'lar eksik
- frontend ve dokumantasyon halen seeded/demo kurguya bagimli

Sonuc: Bu proje teknik olarak urunlesebilir, ancak bugun haliyle "SaaS'a cikmaya hazir" degil. Once platformizasyon ve guvenlik/operasyon katmanlari tamamlanmali.

Guclu Taraflar

- Backend tarafinda company-scoped veri modeli mevcut. `company`, `app_user`, `survey`, `operation`, `call_job`, `survey_response` gibi cekirdek tablolar dogru yone gidiyor. Bkz. *V2__create_company_and_app_user.sql*, *V5__create_call_tables.sql*
- Request seviyesinde company izolasyonu icin interceptor tabanli kontrol var. Bkz. *AuthInterceptor.java*
- Session tabanli auth ve session tablosu mevcut. Bkz. *AuthService.java*, *V10__create_auth_session.sql*

- Survey import, operation execution, webhook ingestion ve response ingestion taraflari urun cekirdegi olarak anlamli bir derinlige sahip.
- Test suiti tamamen bos degil; domain/service seviyesinde anlamli sayida test var. `mvn test` calistirildiginda cok sayida servis testi geciyor, ancak WebMvc tarafinda kirmizilar bulunuyor.

Kritik Bosluklar

1. Self-service SaaS onboarding yok

Mevcut sistemde yeni bir tenant/company olusturma, ilk owner kullanicisini acma, email dogrulama veya davet akisi bulunmuyor. Acik bir `CompanyController`, signup, invitation veya workspace provisioning akisi yok. Auth sadece login/me/logout sunuyor. Bkz. *AuthController.java*

Etkisi:

- her yeni musteri icin manual provisioning gerekir
- urun self-serve satilamaz
- sales-led onboarding disinda buyume zorlasir

Gerekli isler:

- tenant/workspace olusturma API ve UI
- ilk owner olusturma
- invitation kabul akisi
- sifre belirleme / reset
- email verification
- trial baslatma akisi

2. Rol bazli yetkilendirme tanimli ama uygulanmiyor

`OWNER`, `ADMIN`, `ANALYST`, `OPERATOR`, `MEMBER` rolleri enum olarak var; fakat controller/service seviyesinde bu rollere gore bir authorization enforcement gorunmuyor. Spring Security, `@PreAuthorize`, `SecurityFilterChain` veya benzeri bir katman da yok. Bkz. *AppUserRole.java*

Etkisi:

- tenant icindeki tum authenticated kullanicilarin ayni guce sahip olma riski var
- enterprise musteri beklentisi karsilanmaz
- veri degisikligi, operasyon baslatma, survey publish gibi kritik aksiyonlar korunmaz

Gerekli isler:

- endpoint bazli RBAC matrisi
- UI permission gating
- auditlenebilir authorization karar mekanizmasi
- tercihen Spring Security tabanli policy enforcement

3. Faturalama, plan ve kullanim yonetimi yok

Kod tabaninda subscription, billing, usage meter, quota, invoice, payment webhook veya plan enforcement katmani bulunmuyor.

Etkisi:

- gelir modeli teknik olarak uygulanamaz
- kullanima dayali maliyetler kontrolsuz kalir
- trial, soft limit, hard limit, overage gibi SaaS mekanikleri yoktur

Gerekli isler:

- plan modeli: trial / starter / growth / enterprise
- kullanim metrikleri: call sayisi, dakika, aktif operation, storage, export
- billing provider entegrasyonu
- webhook bazli subscription state senkronizasyonu
- payment failure / grace period / suspension akislari

4. Guvenlik varsayimlari production icin uygun degil

Asagidaki noktalar kritik:

- gercek API key ve secret benzeri degerler `application.yml` icinde default olarak duruyor. Bkz. *application.yml*
- auth cookie `secure(false)` olarak set ediliyor. Bkz. *AuthCookieService.java*
- seeded kullanici ve sifre uygulama baslangicinda otomatik yaratiliyor. Bkz. *DataInitializer.java*
- login ekrani varsayilan test credentials ile geliyor. Bkz. *frontend/app/login/page.tsx*
- frontend tarafinda sabit company id kullanimi var. Bkz. *frontend/lib/company.ts*

Etkisi:

- secret leakage riski
- prod ortaminda yanlis konfigurasyonla canliya cikma riski
- tenant baglaminin UI tarafinda dogru tasinmasi zayif

Gerekli isler:

- tum secret'larin vault / env / secret manager'a alinmasi
- secure cookie + HTTPS zorunlulugu
- seed data'nin sadece local/dev profile ile sinirlanmasi
- frontend'de hardcoded company baglaminin kaldirilmasi
- auth hardening: login rate limit, brute-force lockout, session revocation, device/session listesi

5. Multi-tenant yasam dongusu eksik

Veri modeli company-scoped olsa da bir SaaS icin gereken tenant lifecycle yonetimi eksik:

- tenant create/suspend/reactivate/delete
- tenant-level config paneli
- plan bilgisi
- data retention policy

- tenant export / offboarding

`CompanyStatus` enum mevcut ama bunu yoneten bir admin/business akisi gorunmuyor. Bkz. *CompanyStatus.java*

6. Admin/backoffice katmani yok

SaaS operasyonu icin urun disi ama isletme icin zorunlu olan panel/ara yuzler yok:

- tenant listesi
- fatura ve plan gorunumu
- destek amacli impersonation veya support session
- operasyonel incident ekranlari
- provider hata takip ekranlari
- kullanici, invitation ve audit log yonetimi

Bugun urun UI daha cok tenant-icinde operasyon yapmaya odakli.

7. Audit log ve uyumluluk eksik

CRUD ve operasyon akisleri var; ancak kim neyi ne zaman degistirdi, hangi survey publish edildi, hangi kullanici hangi response'u manuel editledi gibi SaaS ve enterprise icin cok kritik audit trail katmani gorunmuyor.

Ozellikle kritik aksiyonlar:

- survey publish/unpublish
- operation start/pause/resume
- call job retry/redial
- survey response manuel guncelleme
- kullanici yetki degisikligi
- billing state degisikligi

Gerekli isler:

- immutable audit event tablosu
- actor, tenant, action, resource, diff metadata
- admin/export ekranlari

Yuksek Oncelikli Platform Bosluklari

8. CI/CD ve release standardizasyonu eksik

Repo icinde workflow, pipeline, deploy manifestleri veya ortam ayrimi icin yeterli release otomasyonu gorunmuyor. Dockerfile ve `docker-compose.yml` var ama bu local deployment seviyesinde. Bkz. *docker-compose.yml*, *Dockerfile*

Gerekli isler:

- PR pipeline: lint, unit test, integration test, build
- environment promotion: dev / staging / prod

- database migration guardrail
- rollback stratejisi
- image versioning ve release tagging

9. Gozlemlenebilirlik production seviyesi degil

Loglar var ama metrik, tracing, alerting ve health-governance katmani gorunmuyor. `actuator`, Prometheus, OpenTelemetry, Sentry vb. bulunmuyor.

Etkisi:

- canli problem oldugunda MTTR yuksek olur
- provider kaynakli hata ve timeout'lar sistematik izlenemez
- tenant bazli SLA takibi yapilamaz

Gerekli isler:

- health/readiness/liveness endpointleri
- metrics: request latency, error rate, webhook lag, dispatch success, provider failure rate
- tracing: request -> operation -> call_job -> attempt -> webhook zinciri
- alerting: payment failure, webhook failure spike, queue backlog, ingestion error

10. Queue/worker mimarisi sinirli gorunuyor

Domain tarafinda job modeli var ama kurumsal SaaS olgunlugunda beklenen ayrik queue/worker altyapisi gorunmuyor. Uygulama ici schedule/dispatch mantigi var; buyuk hacimde tenant trafigi geldiginde yatay olceklenebilir worker tasarimi gerekir.

Gerekli isler:

- gercek queue altyapisi
- worker ownership/locking strategy
- retry ve dead-letter governance
- tenant-aware rate control

Orta Oncelikli Urun Bosluklari

11. Dokumantasyon demo/seed bagimli

README neredeyse bos. Bkz. *README.md*

Mevcut Postman/OpenAPI dokumanlari seeded company id ve seeded veriye referans veriyor. Bu, urunun halen demo moduna yakin oldugunu gosteriyor. Bkz. *docs/openapi-postman-tests.yaml*

Gerekli isler:

- gerÃ§ek onboarding dokumani
- env var referansi
- tenant admin guide
- support runbook
- provider setup guide

- security and compliance guide

12. Frontend tenant-aware ve production-ready degil

Frontend auth var ama halen demo bagimliliklari mevcut:

- hardcoded company id
- varsayilan login bilgileri
- seeded/dummy kurgulara yakin copy ve state

Bu, cok tenantli ve self-service SaaS deneyimini zayiflatiyor.

Gerekli isler:

- aktif workspace'i auth context'ten tam dogru tasima
- multi-workspace support gerekiyorsa workspace switch
- invitation / billing / settings / team management sayfalari
- empty state'lerin demo dilinden cikarilmasi

13. Kullanici ve ekip yonetimi eksik

Bir SaaS urunde asagidakiler beklenir:

- ekip uyeleri listesi
- davet etme
- rol degistirme
- kullanici devre disi birakma
- son giris / aktif sessionlar

Veri modeli bunu destekleyebilecek temeli atmis, ama urun akisi yok.

14. Uyumluluk ve veri yonetimi eksik

Ozellikle sesli anket ve transcript toplandigi icin uyumluluk daha da kritik:

- KVKK/GDPR aydinlatma ve consent akislari
- data retention ve deletion policy
- transcript ve raw payload saklama politikasi
- PII minimization / masking
- DPA / subprocessor / audit readiness

Kodda transcript ve raw payload alanlari var, ama governance tarafi gorunmuyor. Bkz. *V5__create_call_tables.sql*

Kalite ve Release Riskleri

15. Testler tamamen yesil degil

`mvn test -q` calistirildiginda tum suite yesil donmuyor.

Gozlenen problemler:

- `AuthControllerWebMvcTest` Java 25 + Byte Buddy/Mockito uyumsuzlugu nedeniyle context kuramiyor
- `SurveyCorsIntegrationTest` eksik mock bean (`ImportedSurveyOperationService`) nedeniyle context kuramiyor

Bu durum release confidence'i dusurur; SaaS'a cikista "green main branch" kritik gerekliliktir.

16. Local/prod ayrimi daha net olmali

`application-local.yml` ve `application-dev.yml` minimal; ancak prod profile, secret strategy, log policy, cookie policy, provider mode guardrail ve staging davranislari daha net ayrilmali. Bkz. *application-local.yml*, *application-dev.yml*

17. Maliyet kontrol mekanizmalari eksik

Bu urun call provider ve AI provider kullaniyor. SaaS'ta en kritik teknik konulardan biri tenant bazli maliyet izolasyonudur.

Eksik gorunen alanlar:

- tenant bazli spending limit
- provider bazli budget cap
- daily/monthly usage cap
- anomaly detection
- auto-pause on excessive spend

Onerilen Yol Haritasi

Faz 1: SaaS Temeli

- secret cleanup ve config hardening
- seed/demo bagimliliklarini ayirma
- RBAC implementasyonu
- tenant/company management API
- team/invitation/password reset akislari
- green CI pipeline

Faz 2: Satin Alinabilir Urun

- billing/subscription altyapisi
- usage metering ve quota enforcement
- tenant settings ve workspace admin UI
- audit log
- production observability

Faz 3: Operasyonel Olgunluk

- queue/worker olcekleme
- support tooling

- compliance ve retention politikasi
- SSO/SAML/enterprise readiness
- advanced analytics export ve tenant reporting

Onerilen Oncelik Sirasi

En kritik ve hemen ele alinmasi gerekenler:

1. secret ve auth hardening

2. self-service onboarding + tenant lifecycle

3. RBAC ve kullanici yonetimi

4. billing/usage/quota katmani

5. CI/CD + test suite stabilizasyonu

6. observability + audit log

Son Hukum

SurveyAI'nin cekirdek urun mantigi umut verici ve dogru yonde. Ozellikle anket, operasyon, cagri isleme ve response ingestion taraflari "SaaS olacak urunun cekirdegi" olarak degerli. Ancak bugunku haliyle bu proje daha cok:

- demo / pilot
- tek tenantli veya manuel onboardingli beta
- destekle isletilen erken asama urun

sinifinda.

Ticari SaaS'a cikis icin gerekli ana is, yeni ozellik eklemekten cok "platform katmani" insa etmek: authz, tenant lifecycle, billing, audit, observability, ops ve guvenlik.

Inceleme Notlari

Bu degerlendirme repo icindeki mevcut kod ve konfigurasyon uzerinden yapilmistir. Ozellikle canli altyapi, harici CI sistemi veya repo disi operasyon runbook'lari varsa burada gorunmeyen ek olgunluk unsurlari olabilir.